🚀 Space Conquest Online

Version du document : 1.1 (07/04/2026)
Version log :

- 1.0 (01/04/2026) : Version initiale du sujet.
- 1.1 (07/04/2026) : Ajout de précisions, corrections et reformulations.

L'examen

Au fil de ce projet, vous serez amené à développer une application mobile servant de client pour le jeu de stratégie en ligne "Space Conquest Online". Ce projet vous permettra de mettre en pratique l'ensemble des compétences acquises lors du cours d'introduction au développement mobile *cross-platform* avec React Native et Expo.

Informations générales

- **Date de rendu** : 07/04/2026 à 23h59
- **Format de rendu** : Un dépôt GitHub contenant l'intégralité du code source, un fichier `README.md` peut accompagner le projet pour apporter toute les informations complémentaires utiles.
- **Equipe** : Projet individuel
- **Evaluation** : 50% évalués sur le rendu et 50% évalués lors de la code review.

Contraintes techniques

Les contraintes techniques de ce projet sont les suivantes :

- Le projet doit **obligatoirement** être développé avec React Native et Expo.
- Le code doit **obligatoirement** être écrit en TypeScript.
- L'application peut avoir été testée uniquement sur iOS ou Android, mais pas nécessairement les deux.
- L''IA pour la génération de code, bien qu'autorisée, doit être utilisée de manière responsable et ne doit pas être la source principale du code de l'application. L'accent doit être mis sur la compréhension et l'apprentissage du développement mobile, plutôt que sur la simple génération

de code. ⚠️ **Vous devrez être en mesure d'expliquer, de justifier et de parler de l'intégralité de votre code lors de la code review.** ⚠️

Attendus généraux

Lors de la réalisation du projet, il sera attendu que :

- L'application s'exécute et réponde aux exigences du sujet.
- Les bonnes pratiques abordées en cours soient respectées, notamment en ce qui concerne l'architecture de l'application, la gestion de l'état, la structuration du code, les conventions de nommage, etc.
- Le code soit de qualité, lisible et bien organisé, avec des commentaires pertinents lorsque nécessaire.

⚠️ **Toute triche par plagiat ou duplication de code du projet d'un.e camarade de promo entraînera un malus important.**

Conseils de réalisation

Ce sujet inclut un déroulé en étapes suggérées pour vous aider à organiser votre travail selon un avancement logique. Toutefois, vous êtes libre de suivre votre propre approche si vous le souhaitez.

En revanche, il est absolument nécessaire de bien **lire le sujet dans son intégralité** avant de commencer à développer, afin d'avoir une vision complète des exigences et des fonctionnalités à implémenter.

Notez également **que la qualité domine sur la quantité**. Il est préférable de livrer une application fonctionnelle avec un nombre limité de fonctionnalités mais architecturée et développée proprement, plutôt qu'une application incomplète ou non fonctionnelle avec un grand nombre de fonctionnalités.

Le sujet

Contexte

"Space Conquest Online" est un jeu de stratégie en ligne au tour par tour dans lequel les joueurs contrôlent des flottes spatiales et s'affrontent pour conquérir un territoire minier. Chaque joueur peut effectuer différentes actions pendant son tour, telles que déplacer ses vaisseaux, attaquer les vaisseaux ennemis, ou encore acheter de nouvelles unités. L'objectif est de dominer le territoire et de vaincre les flottes adverses.

Déroulement type d'une partie

1. **Création de la session de jeu** : Un joueur crée une session de jeu et invite d'autres joueurs à la rejoindre en affichant un QR code contenant le code d'invitation.
2. **Rejoindre la session de jeu** : Les joueurs rejoignent la session de jeu en scannant le QR code (jusqu'à 4 joueurs au total, y compris le créateur de la session). Après avoir rejoint la session, les joueurs peuvent voir les autres joueurs présents dans la session. Tant que la session n'a pas démarré, les joueurs peuvent quitter la session.
3. **Démarrage de la partie** : À tout moment, le créateur de la session de jeu peut démarrer la partie. Une fois la partie démarrée, les joueurs ne peuvent plus quitter la session.
4. **Tour de jeu** : Les joueurs jouent au tour par tour, en tour simultané (mécanique similaire à des jeux comme "Civilization"). À chaque tour, chaque joueur peut effectuer un certain nombre d'actions puis les valider. Une fois que tous les joueurs ont validé leurs actions, le tour est exécuté et les résultats sont affichés à tous les joueurs.
5. **Fin de la partie** : La partie se termine lorsqu'un joueur est le dernier ayant des vaisseaux sur la carte.

API et documentation

Pour implémenter les fonctionnalités, il n'existe aucune mécanique en temps réel comme des Websockets. Vous devrez interagir avec une API REST pour l'ensemble des requêtes et, lorsque nécessaire (précisé dans le sujet), effectuer un *polling HTTP* (requêtes périodiques à intervalle régulier) pour récupérer des informations.

La documentation de l'API est disponible sur https://space-conquest-online.osc-fr1.scalingo.io/api/documentation . **Consultez cette documentation qui est votre référence, le sujet ne contient que des informations partielles.**

Les requêtes API sont à effectuer vers l'URL de base suivante : `https://space-conquest-online.osc-fr1.scalingo.io/api` .

Cahier des charges

Dans la section ci-dessous, le cahier des charges est présenté sous forme d'étapes d'implémentation suggérées. Vous êtes libre de suivre cet ordre ou non. Toutefois, chaque section contient des informations importantes sur les fonctionnalités à implémenter, les endpoints de l'API à utiliser, et les attentes spécifiques pour chaque fonctionnalité. Il est donc essentiel de lire attentivement chaque section avant de commencer à développer.

Attendus généraux

- **Navigation** : L'application doit comporter une navigation fluide et intuitive entre les différents écrans, permettant aux utilisateurs d'accéder facilement à toutes les fonctionnalités du jeu. Une navigation à plusieurs niveaux est attendue, incluant au moins un `TabsNavigator` .
- **Gestion de l'état** : L'application doit utiliser une solution de gestion de l'état appropriée. Les états partagés doivent être gérés de manière efficace en utilisant uniquement la *Context API* en combinaison avec des *reducers* le cas échéant.
- **Appels à l'API** : L'application doit interagir avec l'API REST de manière efficace et sécurisée, en prenant en charge les réponses et les erreurs de manière appropriée. Les appels à l'API doivent être organisés de manière logique, par exemple en regroupant les appels liés à une même fonctionnalité ou ressource. Les requêtes de récupération de données doivent être implémentées sous forme de *hooks*.
- **Stockage sécurisé** : Les données sensibles, telles que les tokens d'authentification, doivent être stockées de manière sécurisée en utilisant des solutions adaptées pour les applications mobiles.
- **Interface utilisateur** : L'interface utilisateur doit être claire, intuitive et réactive, offrant une expérience de jeu agréable. L'affichage de la carte du jeu doit être lisible et permettre aux joueurs de comprendre rapidement la situation de la partie.

Étape 1 : Inscription et connexion

Objectif : Dans cette première étape, vous devez implémenter les fonctionnalités d'inscription et de connexion des utilisateurs. Les joueurs doivent pouvoir créer un compte en fournissant un nom d'utilisateur, une adresse e-mail et un mot de passe, puis se connecter à l'application pour accéder aux fonctionnalités du jeu.

A réaliser :

- Créer les interfaces d'inscription et de connexion.
- Mettre en place une structure de navigation et de gestion des états (locaux et partagés) adaptées.
- Implémenter les appels à l'API pour l'inscription et la connexion, en gérant les réponses et les erreurs de manière appropriée.
- Stocker de manière sécurisée le token d'authentification reçu lors de la connexion.

Attendus :

- Les utilisateurs doivent pouvoir s'inscrire et se connecter avec succès.
- Les erreurs d'inscription et de connexion doivent être gérées et affichées de manière claire à l'utilisateur.

- Le token d'authentification doit être stocké de manière sécurisée et utilisé pour les requêtes nécessitant une authentification.
- Les utilisateurs non-connectés ne doivent pas pouvoir accéder au reste de l'application.

Informations complémentaires :

- À cette étape, vous pouvez utiliser uniquement l'access token pour authentifier les requêtes à l'API. L'utilisation du refresh token est un bonus détaillé dans la section "Bonus" à la fin du sujet et n'est pas obligatoire pour la validation de l'étape 1.
- L'API de connexion retourne un token d'authentification (access token) qui doit être inclus dans les en-têtes des requêtes nécessitant une authentification (voir documentation de l'API). Ce token a une durée de vie limitée (24 heures).

Étape 2 : Créer, rejoindre ou quitter une session de jeu

Objectif : Dans cette étape, vous devez implémenter les fonctionnalités de création et de gestion des sessions de jeu, ainsi que la possibilité pour les joueurs de rejoindre une session en scannant un QR code.

A réaliser :

- Créer une interface de création de session de jeu, permettant à un joueur de créer une nouvelle session et d'obtenir un QR code d'invitation. **Une session ne peut être rejointe que via l'utilisation d'un QR code.**
- Créer une interface de scan de QR code, permettant à un joueur de rejoindre une session de jeu en scannant le QR code d'invitation.
- Créer une interface de présentation de la session de jeu dans laquelle se situe le joueur. Cette interface doit afficher la liste des joueurs présents dans la session et le nom de la session. Cet écran doit rester à jour.
- Sur l'interface de présentation de la session de jeu, implémenter une fonctionnalité permettant à un joueur de quitter la session de jeu tant que la partie n'a pas démarré. **Le créateur de la session ne peut pas quitter la session, il devra supprimer la partie (voir section suivante).**
- Implémenter les appels à l'API correspondant en gérant les réponses et les erreurs de manière appropriée.

Attendus :

- Les joueurs doivent pouvoir créer une session de jeu et obtenir un QR code d'invitation.
- Les joueurs doivent pouvoir rejoindre une session de jeu en scannant le QR code d'invitation.

- L'interface de présentation de la session de jeu doit afficher les informations de la session et la liste des joueurs présents, et rester à jour en cas de changement (nouveau joueur rejoignant la session, joueur quittant la session, etc.). Pour cela, vous pouvez implémenter un *polling HTTP* pour récupérer périodiquement les informations de la session, à l'intervalle de 30 secondes. Le *polling HTTP* doit être implémenté de manière efficace, en évitant les requêtes redondantes ou inutiles, et en respectant les bonnes pratiques pour la gestion des ressources et de la batterie sur les appareils mobiles.
- Les erreurs retournées par l'API doivent être gérées et affichées de manière claire à l'utilisateur.

Informations complémentaires :

- Les sessions sont limitées à 4 joueurs par le serveur.
- Un *rate limiting* est appliqué par le serveur pour la récupération des informations de la session, avec une limite de 20 requêtes par minute. Assurez-vous de respecter cette limite pour éviter que vos requêtes ne soient bloquées.

Étape 3 : Modération du salon d'attente et démarrage de la partie

Objectif : Dans cette étape, vous devez implémenter les fonctionnalités de modération du salon d'attente par le créateur de la session, ainsi que la possibilité de démarrer la partie.

A réaliser :

- Sur l'interface de présentation de la session de jeu, implémenter une fonctionnalité permettant au créateur de la session de jeu de supprimer la session, ce qui entraîne la suppression immédiate de la session côté serveur. Le cas doit être traité de manière appropriée pour les autres joueurs présents dans la session (affichage d'un message indiquant que la session a été supprimée et redirection vers l'écran d'accueil).
- Sur l'interface de présentation de la session de jeu, implémenter une fonctionnalité permettant au créateur de la session de jeu d'expulser un joueur de la session (kick), ce qui entraîne le retrait immédiat du joueur de la session côté serveur. Le joueur expulsé ne sera pas notifié mais n'aura simplement plus accès à la session et devra (comme dans le cas de la suppression) être redirigé vers l'écran d'accueil. Il peut à nouveau rejoindre la session s'il le souhaite, tant que la partie n'a pas démarré, en scannant à nouveau le QR code d'invitation.
- Sur l'interface de présentation de la session de jeu, implémenter une fonctionnalité permettant au créateur de la session de jeu de bannir un joueur de la session (ban), ce qui entraîne le retrait immédiat du joueur de la session côté serveur et l'impossibilité pour ce joueur de rejoindre à nouveau la session. Le joueur banni ne sera pas notifié mais n'aura simplement plus accès à la

session et devra (comme dans le cas de la suppression) être redirigé vers l'écran d'accueil. Il ne pourra pas rejoindre à nouveau la session même s'il scanne à nouveau le QR code d'invitation.
- Sur l'interface de présentation de la session de jeu, implémenter une fonctionnalité permettant au créateur de la session de jeu de démarrer la partie. Une fois la partie démarrée, il est impossible de quitter la partie ou d'effectuer des actions de modération (kick, ban, suppression de la session). Les joueurs sont redirigés vers les écrans de jeu (voir section suivante).
- Implémenter les appels à l'API corrspondant en gérant les réponses et les erreurs de manière appropriée.

Attendus :

- Le créateur de la session doit pouvoir supprimer la session. Le cas est géré pour l'ensemble des joueurs présents dans la session, qui sont redirigés vers l'écran d'accueil avec un message indiquant que la session a été supprimée.
- Le créateur de la session doit pouvoir expulser un joueur de la session. Le cas est géré pour le joueur expulsé, qui est redirigé vers l'écran d'accueil sans notification spécifique.
- Le créateur de la session doit pouvoir bannir un joueur de la session. Le cas est géré pour le joueur banni, qui est redirigé vers l'écran d'accueil sans notification spécifique et ne peut plus rejoindre la session même s'il scanne à nouveau le QR code d'invitation.
- Le créateur de la session doit pouvoir démarrer la partie. Une fois la partie démarrée, les joueurs ne peuvent plus quitter la session et sont redirigés vers les écrans de jeu.

Informations complémentaires :

- Seul le créateur de la session a accès aux fonctionnalités de modération (kick, ban, suppression de la session) et de démarrage de la partie. Ces vérifications sont effectuées côté serveur, mais assurez-vous de ne pas afficher les boutons correspondants aux joueurs qui ne sont pas le créateur de la session pour une meilleure expérience utilisateur.
- Après le démarrage de la partie, la propriété `state` de la session de jeu passe de `waiting` à `running`. Vous pouvez utiliser cette information pour rediriger les joueurs vers les écrans de jeu lorsque la partie démarre.
- Après le démarrage de la partie, les joueurs ne peuvent plus quitter la session et la modération n'est plus possible. Ces vérifications sont effectuées côté serveur.

Étape 4 : Affichage de la carte et de l'état de la partie

Objectif : Dans cette étape, vous devez implémenter les fonctionnalités d'affichage de la carte du jeu et de l'état de la partie pour les joueurs.

A réaliser :

- Créer une interface de jeu affichant la carte du jeu, sous forme de grille.
- Implémenter l'appel API de récupération de la carte du jeu, une seule fois au chargement de l'interface de jeu. Les données de la carte ne changent pas pendant la partie.
- Implémenter un appel API de récupération initial de l'état de la partie.

Attendus :

- La carte du jeu doit être affichée de manière simple et lisible sous forme de grille, avec une représentation visuelle des différentes entités (vaisseaux et ressources).

Informations complémentaires :

- La carte du jeu est une **grille** dont la taille est définie par les propriétés `width` et `height`. Le coin supérieur gauche de la carte correspond à la position `x: 0` et `y: 0`. La carte possède également une propriété `resource_nodes` contenant les coordonnées des cases de ressources.
- L'état de la partie contient :
  - `round` : le numéro du tour en cours
  - `resource.ore` : la quantité de minerai du joueur
  - `stats` : les statistiques cumulées du joueur depuis le début de la partie
  - `ships` : la liste des vaisseaux sur la carte
  - `round_actions_submitted` : `true` si le joueur a déjà validé ses actions pour le tour en cours (voir section suivante), `false` sinon
  - `status` : `running` si la partie est en cours, `finished` si la partie est terminée (victoire d'un des joueurs)
- Pour afficher la grille **il est très fortement recommandé d'utiliser uniquement un composant FlatList**. Ne vous lancez pas dans l'intégration d'une bibliothèque complexe de rendu 2D ou 3D.
- Vous pouvez utiliser les images ou formes/couleurs de votre choix pour représenter les éléments sur la carte. ⚠️ **Pour des raisons de performances avec FlatList, n'affichez pas une image sur chaque case, mais uniquement sur celles contenant un vaisseau et/ou une ressource.** Privilégiez un fond uni sous la grille pour apporter de la couleur et de l'immersion.

Étape 5 : Actions de jeu

Objectif : Dans cette étape, vous devez implémenter les fonctionnalités permettant aux joueurs d'effectuer des actions pendant leur tour de jeu et inspecter les éléments sur la carte.

A réaliser :

- Rendre les cases de la carte pressables pour permettre aux joueurs d'inspecter les vaisseaux présents. Lorsqu'une case est pressée, les informations du vaisseau doivent être affichées.

- Permettre aux joueurs de sélectionner des vaisseaux qu'ils contrôlent et d'effectuer des actions pendant leur tour de jeu. Les actions disponibles sont les suivantes pour chaque vaisseau :
  - **move** : déplacer le vaisseau vers une case à portée de déplacement
  - **attack** : attaquer un vaisseau ennemi présent sur une case à portée d'attaque
- Ajouter une interface permettant aux joueurs de consulter la liste des types de vaisseaux disponibles à l'achat, avec leurs caractéristiques (points de vie, points d'attaque, portée de déplacement, portée d'attaque, taux de production, coût en minerai).
- Implémenter l'action d'achat d'un vaisseau (dans la limite du minerai disponible). Le joueur devra ensuite sélectionner la case sur laquelle le vaisseau doit être déployé.
- Implémenter un visuel d'attente, une fois les actions de tours validées. Le joueur doit pouvoir continuer à consulter la carte et les informations de la partie en attendant que les autres joueurs valident leurs actions pour le tour en cours. En revanche, il ne peut plus effectuer d'actions de jeu.
- Implémenter les appels API correspondants pour l'exécution des actions de jeu, en gérant les réponses et les erreurs de manière appropriée.
- Implémenter un *polling HTTP* pour récupérer périodiquement l'état de la partie, à l'intervalle de 30 secondes une fois que le joueur a validé ses actions pour le tour en cours. Cette mécanique doit permettre de mettre à jour l'interface avec les résultats du tour une fois que tous les joueurs ont validé leurs actions. Dès que l'état de jeu retourne la propriété `round_actions_submitted` à `false`, le joueur peut à nouveau effectuer des actions de jeu : c'est le tour suivant qui commence. N'oubliez pas de mettre à jour les affichages liés à l'état de jeu (ressources, position des vaisseaux etc...).

Attendus :

- Les joueurs doivent pouvoir inspecter les vaisseaux présents sur la carte en pressant les cases correspondantes. Lors de l'appui sur une case occupée par un vaisseau, les informations de celui-ci doivent être affichées de manière claire.
- Les joueurs doivent pouvoir sélectionner un vaisseau qu'ils contrôlent et effectuer des actions avec ce vaisseau pendant leur tour de jeu, en respectant les règles de portée (attaque et déplacement).
- Les joueurs doivent pouvoir consulter la liste des types de vaisseaux disponibles à l'achat avec leurs caractéristiques. Depuis cette liste, ils doivent pouvoir acheter un ou plusieurs vaisseaux sur la base de leur quantité de minerai disponible. Pour chaque achat, le joueur doit ensuite sélectionner la case sur laquelle le vaisseau doit être déployé, en respectant les règles de placement.
- Une fois les actions de tour validées, les joueurs doivent être informés que leurs actions sont en attente de validation des autres joueurs, et ne doivent plus pouvoir effectuer d'actions de jeu. Ils

doivent cependant pouvoir continuer à consulter la carte et les informations de la partie en attendant.

Informations complémentaires :

- L'ensemble des règles (portée de déplacement, portée d'attaque, coût en minerai, etc.) sont vérifiées côté serveur, mais assurez-vous de ne pas afficher les options d'actions qui ne sont pas disponibles. Si une des actions envoyées est impossible, le serveur retournera une erreur de validation. Il sera nécessaire de corriger le problème et renvoyer une nouvelle requête avec l'ensemble des actions.
- Il n'est possible d'effectuer **qu'une seule action par tour pour chaque vaisseau**. Si plusieurs actions sont envoyées pour un même vaisseau, le serveur retournera une erreur de validation.
- Le gain de ressources est automatique à chaque tour en fonction du nombre de cases ressources sur lesquelles le joueur a placé un vaisseau de type "miner". Le vaisseau produit autant de minerai que l'indique la propriété `gathering_rate` de son type.
- Exemple d'actions envoyées pour un tour de jeu (le format exact des requêtes est à consulter dans la documentation de l'API) :

```
{
    "actions": [
        // Action de déplacement du vaisseau avec l'identifiant 1 vers la position x:4,
        {
            "type": "move",
            "ship_id": 1, // l'identifiant du vaisseau à déplacer, doit être un vaissea
            "target_x": 4, // si la portée du vaisseau est de 2, alors la position x de
            "target_y": 8 // si la portée du vaisseau est de 2, alors la position y de
        },
        // Action d'attaque du vaisseau avec l'identifiant 1 sur le vaisseau ennemi ave
        {
            "type": "attack",
            "ship_id": 2, // l'identifiant du vaisseau attaquant, doit être un vaisseau
            "target_x": 5, // si la portée d'attaque du vaisseau est de 3, alors la pos
            "target_y": 6 // si la portée d'attaque du vaisseau est
        },
        // Action d'achat du vaisseau de type "fighter" et de déploiement sur la positi
        {
            "type": "purchase",
            "ship_type_id": 1, // le type du vaisseau à acheter, doit être présent dans
            "target_x": 2, // la position x de déploiement du vaisseau acheté doit être
            "target_y": 3 // la position y de déploiement du vaisseau acheté doit être
        }
    ]
}
```

- **Règles particulières appliquées par le serveur** :
  - Plusieurs vaisseaux d'un même joueur ne peuvent se déplacer sur la même case pendant un même tour. Cela déclenche une erreur de validation du serveur.
  - Si plusieurs vaisseaux de joueurs différents se déplacent sur la même case pendant un même tour, le vaisseau du joueur ayant validé son tour le plus vite a la priorité. Les autres vaisseaux annuleront leur déplacement.
  - En cas de combat, si un vaisseau est détruit, son attaque sera tout de même appliquée pour le tour en cours. On considère toujours qu'un vaisseau a le temps de riposter lors d'un combat avant d'être détruit. Cela peut entrainer des situations de "trade" dans laquelle deux vaisseaux s'attaquent mutuellement et sont détruits au même tour.
  - Un vaisseau acheté peut être déployé sur une case libre ou sur une case contenant une ressource. Il ne peut pas être déployé sur une case contenant un autre vaisseau. Si la case de déploiement devient occupée par un autre vaisseau avant l'achat (par priorité d'un autre joueur), l'achat est annulé et le joueur n'est pas débité du minerai correspondant.

Étape 5 : Elimination des joueurs et fin de la partie

Objectif : Dans cette étape, vous devez implémenter les fonctionnalités liées à l'élimination des joueurs et à la fin de la partie.

A réaliser :

- Implémentater la gestion de l'élimination d'un joueur lorsque tous ses vaisseaux sont détruits. Le joueur éliminé peut continuer à consulter la carte et les informations de la partie, mais ne peut plus effectuer d'actions de jeu. Le joueur éliminé doit être informé de son élimination par l'affichage d'un message clair.
- Implémenter la gestion de la fin de la partie lorsque tous les joueurs sauf un sont éliminés. La propriété `status` de l'état de jeu passe à `finished`. Tous les joueurs sont redirigés vers un écran de fin de partie affichant le nom du joueur gagnant, le classement et les statistiques de la partie.
- Créer la vue de fin de partie affichant le nom du joueur gagnant et les statistiques de la partie.
- Implémenter les appels API correspondants pour la gestion de l'élimination des joueurs et de la fin de la partie, en gérant les réponses et les erreurs de manière appropriée.

Attendus :

- Lorsqu'un joueur est éliminé, il doit être informé de son élimination par l'affichage d'un message clair, et ne doit plus pouvoir effectuer d'actions de jeu.
- Lorsqu'un joueur est éliminé, il doit pouvoir continuer à consulter la carte et les informations de la partie.
- Lorsqu'une partie se termine, tous les joueurs doivent être redirigés vers un écran de fin de partie affichant le nom du joueur gagnant, le classement et les statistiques de la partie.
- Les erreurs retournées par l'API doivent être gérées et affichées de manière claire à l'utilisateur.

Etape 6 : Profil et historique des parties

Objectif : Dans cette étape, vous devez implémenter les fonctionnalités liées au profil de l'utilisateur et à l'historique de ses parties.

A réaliser :

- Créer une interface de profil de l'utilisateur affichant ses informations (nom et adresse e-mail).
- Créer une interface d'historique des parties de l'utilisateur affichant la liste des parties auxquelles l'utilisateur a participé et permettant d'en consulter les détails.

- Créer une interface de modification des informations du profil de l'utilisateur, permettant à l'utilisateur de modifier son nom, son adresse e-mail et son mot de passe.
- Ajouter la fonctionnalité de déconnexion de l'utilisateur.
- Implémenter les appels API correspondants pour la gestion du profil de l'utilisateur et de son historique de parties, en gérant les réponses et les erreurs de manière appropriée.

Attendus :

- L'interface de profil de l'utilisateur doit afficher les informations de l'utilisateur.
- L'interface d'historique des parties de l'utilisateur doit afficher la liste des parties auxquelles l'utilisateur a participé et permettre d'en consulter les détails.
- L'interface de modification des informations du profil de l'utilisateur doit permettre à l'utilisateur de modifier son nom, son adresse e-mail et son mot de passe, en gérant les erreurs de validation et en affichant les messages d'erreur de manière claire à l'utilisateur.
- La fonctionnalité de déconnexion doit permettre à l'utilisateur de se déconnecter de l'application, en supprimant le token d'authentification stocké et en redirigeant l'utilisateur vers l'écran de connexion.
- Les erreurs retournées par l'API doivent être gérées et affichées de manière claire à l'utilisateur.

Bonus : Utilisation du refresh token pour la gestion de l'authentification

Objectif : Implémenter la gestion du refresh token pour permettre une expérience utilisateur plus fluide en évitant de déconnecter l'utilisateur lorsque le token d'authentification expire.

A réaliser :

- Lors de la connexion, stocker de manière sécurisée le refresh token en plus de l'access token.
- Implémenter une logique de rafraîchissement du token d'authentification lorsque une requête à l'API retourne une erreur d'authentification indiquant que le token a expiré. Cette logique doit envoyer une requête de rafraîchissement du token en utilisant le refresh token, et en cas de succès, stocker le nouveau token d'authentification puis relancer la requête initiale ayant échoué.
- Gérer les cas d'erreur lors du rafraîchissement du token. Par exemple : si le refresh token a également expiré ou est invalide, l'utilisateur est redirigé vers l'écran de connexion.

Attendus :

- Le refresh token doit être stocké de manière sécurisée.
- Lorsqu'une requête à l'API retourne une erreur d'authentification indiquant que le token a expiré, la logique de rafraîchissement du token doit être déclenchée. En cas de succès, la requête

initiale doit être renvoyées avec le nouveau token d'authentification.
- En cas d'erreur lors du rafraîchissement du token, l'utilisateur doit être redirigé vers l'écran de connexion.

Bonus : Retour haptique et vibrations

Objectif : Ajouter des retours haptiques et des vibrations pour améliorer l'expérience utilisateur lors de certaines actions ou événements dans le jeu.

A réaliser :

- Ajouter des retours haptiques lors de certaines actions de jeu, par exemple lors de la validation des actions du tour.
- Ajouter des vibrations lors de certains événements, par exemple lorsqu'un vaisseau est détruit.

Conclusion

Ce projet vous permettra de mettre en pratique l'ensemble des compétences acquises lors du cours d'introduction au développement mobile avec React Native et Expo, en développant une application mobile complète pour un jeu de stratégie multijoueur.

Ce sujet est volontairement complexe et conséquent. Il est important de bien planifier votre travail, de lire attentivement les exigences et de procéder par étapes. Pour toute question, remarque ou signalement de bug dans l'API, n'hésitez pas à contacter votre intervenant sur Discord ou par email. Bonne chance et bon courage !